从仿真器到训练栈

Isaac Lab ·MuJoCo ·mjlab ·UniLab ·Newton ·Genesis World

仿真框架没有洗牌，机器人训练栈正在分层

一份关于六个项目、六层位置与一条闭环的成本分析

研究对象	Isaac Lab • MuJoCo • Playground mjlab • UniLab • Newton • Genesis World
核心判断	竞争单位从单次仿真速度，转向每次有效验证的返工成本
方法	分层框架 + 横向对比 + 成本模型 + 组合路线
资料口径	论文、官方文档、代码仓库与厂商博客；截至 2026-10-28

摘要

有了 Isaac Lab 和 MuJoCo，为什么还会出现 mjlab、UniLab、Newton、Genesis？本文的回答是：这些项目并不在同一个位置上竞争。Isaac Lab 做平台，MuJoCo 做物理基座，MuJoCo Playground 与 mjlab 做任务入口，UniLab 做运行时，Newton 做引擎底座，Genesis World 做闭环评估。它们共同指向同一个工程问题：机器人学习最贵的地方，正在从**单次仿真速度**转向**整条训练与评估闭环的返工成本**。

本文首先给出一个六层训练栈框架，把六个项目放回各自的位置；随后逐个剖析其架构、关键设计、代价与边界；在此基础上提出两个简单的成本模型——time-to-robot 与每次有效验证的成本——并用它们解释为什么这些项目会同时出现。本文认为，真正的分水岭不是「谁的物理引擎更强」，而是「谁能让真实失败更快、更稳定地回到训练系统」。最后给出三套组合路线、一份反模式清单与未来 12 个月的观察指标。

关键词：机器人学习；仿真；Sim2Real；训练栈；物理引擎；闭环评估

目录

摘要 **i**

1 引言：一个被问错的问题 **1**

1.1 “谁更强”不是一个好问题 1

1.2 两种常见叙事，以及它们各自漏掉了什么 1

1.3 本文的判断 1

1.4 结构 2

2 分析框架：六层训练栈 **2**

2.1 仿真其实在扮演三种角色 2

2.2 六层模型 2

2.3 两个成本模型 3

2.4 读图的三个注意 4

3 六个项目：各自替谁付账 **4**

3.1 Isaac Lab：把复杂性收进同一个工作台 4

3.2 MuJoCo 与 Playground：可读的物理，和被压短的 time-to-robot 6

3.3 mjlab：环境定义本身成为基础设施 7

3.4 UniLab：瓶颈藏在仿真器与算法之间 7

3.5 Newton：底层物理引擎，也是连接器 8

3.6 Genesis World：把仿真做成评估系统 9

4 横向比较：位置、能力与选择 **10**

4.1 先比位置，再比能力 10

4.2 能力与约束 10

4.3 决策：先看最贵的返工项 12

5 训练栈的系统经济学 **12**

5.1 返工从哪里来 12

5.2 time-to-robot 的分解 13

5.3 评估有效性：分数为什么会骗人 13
5.4 接口稳定性与锁定 13

6 组合路线：不要只押一个名字 14
6.1 三套组合 14
6.2 迁移路径 14
6.3 反模式 15
6.4 一个最小可行闭环 15

7 判断与观察清单 15
7.1 六个判断 15
7.2 未来 12 个月看什么 16
7.3 风险 16
7.4 对无人机读者的映射 17

8 结论 17

A 术语表 17

B 资料与口径 17

1 引言：一个被问错的问题

1.1 “谁更强”不是一个好问题

把 Isaac Lab、MuJoCo、mjlab、UniLab、Newton、Genesis 放进一张表，问「谁才是 Sim2Real 的答案」，很容易得到一个看起来正确、实际上没有操作性的结论：Isaac Lab 工程完整，Newton 物理先进，mjlab 轻量好用。这句话不能算错，但它把不同层级的东西放在了同一个坐标轴上。

真实训练里，问题很少这么干净。你在仿真里看到的策略可能已经很顺：步态稳定，速度跟踪不错，曲线也在往上走。但真机一上去，脚底开始打滑，机身出现奇怪的小抖动，接触点和仿真不一样，执行器延迟把原来很稳的动作带偏。这种时候，再问「哪个仿真器最强」已经晚了。更应该问的是：

判断

这个训练系统，能不能把真机暴露出来的问题，再带回下一轮训练里？

1.2 两种常见叙事，以及它们各自漏掉了什么

叙事一：新框架替代旧框架。这种写法把 mjlab 写成「小号 Isaac Lab」，把 Newton 写成「PhysX 的替代品」，把 Genesis 写成「又一个 Genesis」。它的问题是忽略了依赖关系：mjlab 复用 Isaac Lab 的 manager API；Newton 是 Isaac Lab 3.0 的可选后端；Genesis 的评估对象往往是 Isaac Lab / MuJoCo 训练出来的策略。它们不是替代关系，而是上下游关系。

叙事二：只看单点性能。这种写法比较 FPS、比较并行环境数、比较渲染质量。单点性能当然重要，但它解释不了一个现象：为什么一个团队用了更快的仿真器，实验节奏却没有变快？因为瓶颈可能不在物理步进，而在环境搭建、奖励调试、sim2sim、评估排队、真机返工。峰值 FPS 只覆盖了整条回路的一小段。

1.3 本文的判断

判断

仿真框架没有突然洗牌，但机器人训练栈正在从「训练工具」变成覆盖物理、渲染、运行时、评估和真机闭环的基础设施。这轮变化的主角不是「谁替代谁」，而是「谁负责哪一段返工成本」。

这个判断有三个推论。第一，六个项目的价值不能用同一把尺子量；第二，选型应该按「当前最贵的返工项」决定，而不是按「功能列表最长」决定；第三，长期竞争力来自接口与闭环，而不是某个单点指标。

1.4 结构

第 2 节建立六层训练栈框架与两个成本模型；第 3 节逐个剖析六个项目；第 4 节做横向比较与决策树；第 5 节讨论训练栈的系统经济学；第 6 节给出组合路线与反模式；第 7 节给出判断与观察清单；第 8 节总结。

2 分析框架：六层训练栈

2.1 仿真其实在扮演三种角色

讨论仿真时，最容易混淆的是它的角色。至少可以区分三种：

1. **数据生成器**：为模仿学习、世界模型、感知训练生成样本。
2. **训练环境**：为强化学习提供可交互的 MDP，重点是吞吐、并行、随机化。
3. **评估与迭代引擎**：在不依赖真机排队的前提下，对策略做闭环评估、失败定位与下一轮数据决策。

过去大家更关注前两种，第三种常被当成「训练完顺便跑一下」。Genesis World 的叙事把这第三种角色抬到了产品层：如果训练和评估都来自同一个仿真分布，模型表现变好到底是能力变强，还是更适配模拟器，很容易混在一起。角色不同，对仿真器的要求也不同：数据生成看重渲染与资产，训练看重吞吐与并行，评估看重相关性、可复现性与扰动覆盖。

2.2 六层模型

把「仿真」拆成一个栈，可以得到表 1 的六层。这里的层不是严格的自底向上依赖，而是按 **主要价值主张**归类；一个大平台可以纵向穿透多层。

表 1 机器人训练栈的六层模型

层	主要职责	代表	这一层失效时的症状
L1 评估	闭环评估、相关性校准、失败归因、数据回流	Genesis World	分数涨了，真机没涨；失败样本回不到下一轮
L2 入口	环境库、reward/curriculum 组织、快速试错	Playground、mjlab	环境搭一周；改一次 reward 要等半天
L3 运行时	采样/学习解耦、缓冲、同步、后端契约	UniLab	GPU 利用率低；学习器等数据，采样器等参数
L4 平台	资产、场景、多模态传感、训练/部署一体化	Isaac Lab	资产、传感器、部署脚本各写一套，接口冲突
L5 引擎	接触、约束、多物理、可微、求解器	MuJoCo、Newton、PhysX	接触不稳；复杂物理不支持；sim2sim 差异大
L6 计算	GPU 计算、编译器、渲染、场景描述	Warp、Quadrants、Nyx、OpenUSD	跨硬件困难；渲染/编译成为瓶颈



图 1 六层训练栈与代表项目。层按主要价值主张划分，大平台可纵向穿透多层。

2.3 两个成本模型

如果训练栈的价值是「减少返工」，就需要一个能描述返工的成本模型。本文用两个简化公式。

time-to-robot（T_{2R}）

$$T_{2R} = t_{task} + t_{train} + t_{sim2sim} + t_{deploy} + t_{validate} \tag{1}$$

其中 t_{task} 是环境与奖励搭建，t_{train} 是训练墙钟，$t_{sim2sim}$ 是跨引擎/跨模型验证，t_{deploy} 是真机部署与调试，$t_{validate}$ 是评估与置信建立。峰值 FPS 只影响 t_{train} 的一段。

每次有效验证的成本（C_{eff}）

$$C_{eff} = \frac{C_{build} + C_{train} + C_{eval} + C_{real}}{N_{validated}} \tag{2}$$

$N_{validated}$ 是「通过验证、可以进入下一轮」的实验数。分母很关键：如果一个框架让实验跑得很快，但大部分实验因为环境错误、评估不可信或真机不可复现而作废，C_{eff} 并不会下降。

返工乘子 把失败导致的重复实验显式写出来：

$$C_{eff} = \frac{C_0}{(1 - r_{rework})\, N_0}, \qquad 0 \le r_{rework} < 1 \tag{3}$$

r_{rework} 是「因为某一层失效而必须重跑」的比例。训练栈的竞争，本质上是降低 r_{rework}：更好的环境抽象降低奖励设计错误，更好的 sim2sim 降低迁移失败，更好的评估降低「分数虚高」，更好的运行时降低「等出来」的空转。

可复用结论

单点性能优化的是 t_{train}；训练栈优化的是 T_{2R} 与 r_{rework}。这解释了为什么一个更快的仿真器，未必能让研究节奏变快。

2.4 读图的三个注意

第一，**层不是替代关系**。L2 的 mjlab 依赖 L5 的 MuJoCo Warp；L4 的 Isaac Lab 可以选择 L5 的 Newton；L1 的 Genesis 评估的往往是 L4/L2 训练出的策略。

第二，**同一项目可以穿透多层**。Isaac Lab 从 L4 一直吃到 L5/L6；Genesis 从 L1 做到 L6。分类看的是「它主要替谁付账」，不是「它只做那一层」。

第三，**层的边界在移动**。Isaac Lab 3.0 把物理后端变成可选，等于把 L4 与 L5 解耦；UniLab 把运行时从仿真器里拆出来，等于把 L3 独立；Genesis 把评估从训练里拆出来，等于把 L1 产品化。这三次「解耦」是本文认为最值得关注的长期趋势。

3 六个项目：各自替谁付账

这一节按「平台 → 物理/入口 → 入口 → 运行时 → 引擎 → 评估」的顺序展开。每个项目都回答四个问题：它解决什么工程问题；它的架构如何支撑这个目标；它做了什么关键取舍；它在训练栈中的接口是什么。

3.1 Isaac Lab：把复杂性收进同一个工作台

定位 Isaac Lab 是 NVIDIA 对 Isaac Gym 的「自然继任者」。论文把它描述为 GPU 并行物理、照片级渲染、模块化环境设计与训练流程的统一体，并强调执行器模型、多频率传感器、数

据采集、域随机化、强化学习与模仿学习都在同一个可扩展平台里。它卖的不是单点速度，而是**复杂机器人项目的组织能力**。

架构 Isaac Lab 的结构可以概括为四条纵向链路加一个横向选择。纵向链路是：任务与 MDP（manager-based / direct）、资产与场景（OpenUSD；URDF / MJCF importer；rigid / articulation / deformable；terrain）、物理与传感（PhysX / Newton / OVPhysX，camera / ray caster / IMU / contact / frame transformer）、训练与部署（RSL-RL、RL-Games、skrl、SB3、RLinf、TorchRL、LEAPP export）。横向选择是后端预设：`physics=isaacsim_physx`、`physics=newton_mjwarp`、`physics=ovphysx`，以及独立的 renderer 与 visualizer 选择。

3.0 最关键的结构变化是**工厂化的多后端架构**：`Articulation`、`RigidObject`、`ContactSensor` 背后是抽象基类，具体实现放在 `isaaclab_physx`、`isaaclab_newton`、`isaaclab_ov` 等扩展包里。任务代码不需要知道当前跑在哪个物理引擎上，但可以显式选择后端。另一条变化是 Warp-native 数据通路：资产和传感器的 `.data.*` 返回 `ProxyArray`，可以零拷贝地取 `.torch` 或 `.warp` 视图。

关键设计 第一，用 manager 把环境拆成可组合的 term，使速度跟踪、动作模仿、地形行走、移动操作可以共享大量组件。第二，把传感器当成一等公民，而不是附加项；真实人形系统的 IMU、编码器、相机、接触、力矩传感器频率不同、延迟不同，如果框架只返回一个观测向量，后面接 VLA 或世界模型时会反复打补丁。第三，后端可插拔，并提供 PhysX 与 Newton 之间的策略迁移指南与验证示例；这让「同一策略在两个引擎里都跑得通」成为可执行的 sim2sim 检查。第四，kit-less：大多数工作流可以只装 Newton 后端，用 Newton GL、Viser、Rerun 做可视化；需要 ROS bridge、XR、livestreaming、完整 RTX 时再装 Isaac Sim。这把「轻量试一下」和「完整部署」从二选一变成了连续谱。

代价 Isaac Lab 的问题不是功能少，而是获得这些功能的路径长。安装与依赖重，即使 kit-less 降低了门槛，完整 RTX / ROS / XR 仍需 Isaac Sim 与对应的驱动、容器或 wheel；概念多，USD、PhysX schema、scene、manager、preset、backend、renderer、visualizer 每层都有自己的配置；版本迁移有真实成本，2.x 到 3.0 把四元数从 WXYZ 改成 XYZW，数据属性改成 `ProxyArray`，schema 改成 fragment，训练入口统一成 `isaaclab train`；英伟达生态耦合仍然是最顺的路径，跨厂商硬件不是它的设计目标。

口径与限制

「Isaac Lab 太重」只对了一半。更准确的说法是：它把复杂度前置了。如果项目会走到多模态、多任务、多人协作、真机部署，这些复杂度迟早会出现；如果只是验证一个 reward，为它付前置成本并不划算。

栈中接口 Isaac Lab 位于 L4，向上提供任务、训练、评估与部署，向下消费 PhysX / Newton / OVPhysX 与 Isaac Sim / OV / Warp 生态。它是六个项目里唯一一个**纵向整合**程度最高的：既定义任务 API，也定义后端接口，还提供部署工具。这决定了它的比较对象不是 mjlab，而是「一套自建工具链」。

3.2 MuJoCo 与 Playground：可读的物理，和被压短的 time-to-robot

定位 MuJoCo 是机器人动力学、sim2sim 和快速验证的公共基座。它的长期价值不在「比谁快」，而在**你能看懂发生了什么**：MJCF 是文本，模型结构清晰，接触与约束的求解过程可以追踪，仿真数据结构和 Python 侧几乎一一对应。真机出现脚底打滑、接触点偏移、执行器延迟时，你需要的是一个能逐项对照的模型，而不是一个黑箱。

MuJoCo Playground 则是在这个基座上做的一层「薄入口」。它的论文目标很直白：streamlining simulation, training, and sim-to-real transfer。它支持四足、人形、灵巧手、机械臂，支持 state 与 pixel 输入的零样本迁移，把物理（MJX / MJWarp）、批量渲染（MJWarp batch renderer）和训练环境打包成可复现的整体。

架构 四层结构：入口层（dm_control 环境、locomotion、manipulation、vision，Brax PPO / RSL-RL）、批处理与渲染（MJX、MJWarp、batch renderer）、物理引擎（刚体动力学、软约束接触、关节/执行器/传感器、MJCF）、使用姿态（`pip install playground`、单卡分钟级训练、Colab）。它同时支持 MJX 与 MJWarp 两条 GPU 路径，这不是「多一个选项」：MJX 适合已经投入 JAX 生态的团队，MJWarp 则与 Isaac Lab / Newton / mjlab 的 Warp 路线一致。

关键设计：time-to-robot 这是 Playground 最值得借鉴的概念。它把研究者的真实痛点说清楚了：从「我有一个行为想法」到「我在真机上看到这个行为」，中间到底要多久。大多数训练很少一次写对奖励；第一版奖励学出奇怪姿态，第二版修掉一个问题又引入新的投机行为；策略在仿真里能跑，还要过 sim2sim、延迟、执行器、摩擦、随机化和部署。中间每多等一小时，研究者就少一次尝试。Playground 优化的是这条链路的首次验证延迟，而不是峰值 FPS。

代价 场景规模与真实感有限：MuJoCo 的强项是刚体动力学与接触，不是大场景、照片级渲染、复杂材质。部署链路要自己接：真机 SDK、通信、状态估计、安全逻辑仍然由团队负责。接触模型的「透明」也是双刃剑：软约束接触稳定、可微、好调试，但和真实硬接触的差距需要靠建模与随机化补。MJX 与 MJWarp 两条路径的算子覆盖、性能和社区支持并不完全一致，选型时要按任务验证。

栈中接口 MuJoCo 位于 L5，是 mjlab、UniLab、Newton、Isaac Lab 的 Newton 后端共同依赖的物理基座；Playground 位于 L2，是把 L5 的能力包装成「研究者一下午能跑起来」的入口。两者合起来，覆盖了「物理可信」与「首次验证快」两个不同目标。

3.3 mjlab：环境定义本身成为基础设施

定位 mjlab 常被读成「MuJoCo Warp + Isaac Lab API」的一次工程拼装。这个描述没错，但漏掉了重点：它把**环境定义本身**抬成了一个需要长期维护的基础设施问题。人形机器人训练最怕的往往不是训练慢，而是代码越写越乱。今天做速度跟踪，明天做动作模仿，后天加地形，再后来加视觉、负载、末端接触、摔倒恢复；如果每个环境都是一个巨型脚本，短期能跑，长期会很难维护。等你要比较多个机器人、本体版本和奖励设计时，实验会被代码结构反过来拖住。

架构 链路很清楚：实体组成 `MjSpec`，编译成 `MjModel`，交给 MuJoCo Warp 做 GPU 并行仿真；上层用 `ManagerBasedRlEnv` 管理 RL 任务，再接 RSL-RL 训练。manager 层把 observation、reward、event、command、curriculum、metrics 拆成可组合 term。它没有隐藏的「平台层」，研究者可以一路往下看到 MuJoCo 的数据结构。

关键设计 第一，把「开局摩擦」当成一等指标：`uvx --from mjlab --refresh demo` 可以直接跑 demo，训练命令也只差一个 task id；论文把 single command install、minimal dependencies 写进摘要，说明作者认为研究者在前 30 分钟流失是一个真实问题。第二，保留原生 MuJoCo 的可调试性：接触、约束、执行器的问题可以按 MuJoCo 的方式排查，而不是只能看日志。第三，用 dummy agent 做 MDP sanity check：`--agent zero` 发送零动作，`--agent random` 发送均匀随机动作；在正式训练前先确认机器人不会穿地、不会爆炸，再确认 termination、reset、observation 的维度与语义没问题。这把「调试环境」和「调试算法」拆开了。

生态信号 Unitree 的 `unitree_rl_mjlab` 基于 mjlab + MuJoCo，支持 Go2、A2、As2、G1、R1、H1_2、H2，流程写成 Train → Play → Sim2Real，训练时导出 ONNX，部署前用 `unitree_mujoco` 做仿真验证。这个信号比「mjlab 很轻」更重要：轻量框架如果只停留在 toy task，很难证明自己；当机器人厂商拿它组织 RL 流程，说明它至少在「快速研究」和「真实任务」之间找到了接口。

代价 它不是完整平台：没有 Isaac Lab 级别的 USD 资产管理、ROS bridge、XR 遥操作、HIL、照片级渲染；训练依赖 NVIDIA GPU，macOS 只支持评估；传感器、可变形体、多物理耦合、复杂场景的支持不如 Isaac Lab / Newton / Genesis；生态还年轻，第三方任务、资产、教程的数量远少于 Isaac Lab。这些不是缺陷，而是设计选择：它优化的是研究前段，而不是全流程覆盖。

3.4 UniLab：瓶颈藏在仿真器与算法之间

定位 过去几年，机器人 RL 的默认答案是「把物理也推到 GPU 上」。UniLab 没有否定这条路线，而是问了一个更系统的问题：训练慢，到底慢在物理 step，还是慢在整条训练回路？强化学习训练当然包含物理仿真，但整条回路还包括轨迹采样、策略更新、价值函数更新、参数

同步、经验缓存、数据搬运、算法依赖和硬件调度。如果仿真器很快但学习器在等数据；如果学习器很快但采样器在等新参数；如果数据在 CPU/GPU 之间搬运的成本很高——单看物理仿真峰值 FPS 就会误导判断。

架构 CPU 侧批量仿真（MuJoCoUni、MotrixSim）生产数据，GPU 侧学习器消费数据，中间是统一运行时，负责共享内存、环形缓冲、数据流、参数同步与调度。任务语义用 Hydra 的 owner YAML 组织，动作、观测、奖励、终止、事件、命令、课程、指标都是可组装 term；`SimBackend` 把任务状态、动作、观测相关数据、reset、间隔随机化、地形上下文、回放能力定义成统一接口。算法按同步约束组织：PPO 是严格同步，APPO 允许收集与学习重叠，FastSAC / FlashSAC 用 replay 进一步放松依赖。这个分类很有洞察力——它说明**算法选择本身就是一种系统调度选择**。

证据 论文在 RTX 4090 + AMD Ryzen 9 9950X3D + 64 GB 的受控工作站上报告：G1 Flip（PPO）3.3×、G1 Walk Flat（FastSAC）8.4×；项目页给出的 G1 Motion Tracking（FlashSAC）示例达到 11.0×。论文摘要的总体口径是 3–10×，11.0× 可视为代表性上限而非普适承诺。吞吐方面，CPU 批量仿真在复杂接触与灵巧操作上并不必然弱于 GPU：例如 Ryzen 9 9950X3D 上，Go2 任务 MuJoCoUni 约 246k env steps/s，MotrixSim 约 704k；Threadripper 9980X 上分别约 916k 与 2.66M。这组数据的意义不是「CPU 比 GPU 快」，而是 CPU 侧吞吐足以喂饱一个 GPU 学习器，从而让异构拆分在单机工作站上成立。

代价 适用范围是「仿真主导 + 可解耦」：论文明确说，在严格同步或视觉主导的任务上，CPU/GPU 解耦的收益会变小；当前实现只覆盖刚体，不含可变形体、软体、流体；结论有硬件边界，3–10× 是受控的单 CPU/单 GPU 结果，不是极端规模下的峰值；系统复杂度转移了，你不再需要把所有东西塞进 CUDA，但需要理解缓冲、同步、参数广播和 replay 的时序，调错方向的性能问题更难定位。

口径与限制

不要把 UniLab 读成「GPU 仿真没用」。它的结论是「GPU 驻留不是必要条件」，不是「GPU 驻留是错误路线」。在大规模 GPU 集群、复杂视觉任务、已经深度绑定 Isaac 生态的团队里，GPU 驻留仍然合理甚至更优。

3.5 Newton：底层物理引擎，也是连接器

定位 Newton 不是一个「拿来就能跑任务」的框架。它没有 Isaac Lab 的任务管理器，也没有 Playground 的环境库；它提供的是**物理世界的计算底座**：刚体、关节、接触、可变形体、颗粒、布料、线缆，以及把这些求解器组织起来的场景与状态表示。它和上层框架的关系，更像「编译器后端」和「语言前端」：Isaac Lab 3.0 把 Newton 作为可选的物理后端，通过 `physics=newton_mjwarp` 这样的预设切换；同一个任务可以跑在 PhysX 上，也可以跑在 Newton 上，并提供了 PhysX

与 Newton 之间的策略迁移验证。

架构 Newton 建立在 NVIDIA Warp 之上，扩展并泛化了 Warp 已弃用的 `warp.sim` 模块，把 MuJoCo Warp 集成为主要后端，用 OpenUSD 表达场景与资产，强调 GPU 计算、可微性和用户自定义扩展。求解器包括 MuJoCo-Warp、XPBD、Featherstone、VBD、MPM、Kamino 等。它由 Disney Research、Google DeepMind、NVIDIA 共同发起，由 Linux Foundation 管理，代码 Apache-2.0，文档 CC-BY-4.0。

关键信号 第一，治理结构：这不是普通的公司开源项目，而是试图把物理引擎变成多方共同治理的基础设施；对担心单一厂商锁定的团队，这是积极信号，但治理开放不等于技术成熟。第二，多物理：人形机器人早晚会遇到全身接触、灵巧手细粒度接触、移动操作中的推拉拖挤，以及布料、软体、颗粒材料、触觉；Newton 的 MPM、VBD、布料、线缆、软体示例说明它在为这些任务做准备。第三，Isaac Lab 3.0 把它从「实验性集成」变成「一等后端」：`isaaclab_newton` 支持 MuJoCo-Warp、VBD、MPM、Kamino，并且可以在不安装 Isaac Sim 的 kit-less 模式下运行。第四，它把「物理保真度」变成可替换变量：任务 API 不变，物理后端可换，这会让「用哪个求解器」从架构决策变成一个实验变量。

代价 年轻：API、求解器覆盖、文档都在快速变化，Isaac Lab 3.0 的 release notes 仍把部分功能标为 backend-specific 或 unsupported。硬件门槛：GPU 路径需要 NVIDIA Turing 及以上与较新的驱动，macOS 只能 CPU 运行。求解器差异是真实的：MuJoCo-Warp、XPBD、Featherstone、VBD 的行为不同，策略迁移不是「换个 flag 就等价」。生态仍在建设：相比 PhysX / MuJoCo 数十年的积累，Newton 的资产、教程、社区答案都还少。

口径与限制

如果明天只是训练一个普通速度跟踪策略，Newton 未必是第一选择。稳定、资料多、调试顺、能和现有代码接起来，可能比「更新的物理引擎」更重要。Newton 的价值更适合放在 6–18 个月的时间尺度上看。

3.6 Genesis World：把仿真做成评估系统

定位 很多人一提机器人仿真，第一反应是「用它生成训练数据」。Genesis World 的切入点更靠前：**先把评估做好**。它把仿真称为机器人基础模型的「评估与迭代引擎」，并强调只用真实数据训练、在仿真中直接评估——训练与评估分布刻意分离，避免「模型只是更适配仿真器」的假进步。

架构 四层系统栈。Simulation Interface 负责资产解析（URDF / MJCF / OBJ / GLB / USD）、控制器、传感器、并行/异构环境与 GUI；Render 包含 Nyx（自研机器人实时光追）、Luisa（DSL 光追）、Pyrender（光栅化）；Physics 统一刚体、FEM、MPM、PBD/SPH、uipc、Coupler、

SAP，共享同一场景与状态；Compiler 是 Quadrants（Taichi fork），把 Python kernel 编译到 CUDA、ROCm、Metal、Vulkan、x86、ARM64，并承载自动微分、GPU 图与缓存。它超过单个物理求解器，目标不是「更快的刚体引擎」，而是「物理 + 渲染 + 编译 + 评估」的整体。

评估方法论 Genesis 博客的核心论据是：一项原本需要真实机器人连续运行很久的评估，可以在 GPU 集群上用仿真在约半小时内完成；并称仿真评估与真机执行结果的相关性约 89%。公开报道中的具体口径是：一个约 4 万次独立试验的灵巧手抓取评估，真机连续运行约需 166 小时，仿真约 30 分钟；也有转述写作「超过 200 小时到 0.5 小时以内」。口径略有差异，但量级一致：把「等真机排队」变成「用计算批量比较」。

比数字更值得学的是方法论：仿真与真机并排、同初始化；观测可以按层混合（sim / real），用来归因物理、渲染、控制、通信的 gap；沿视觉、行为、语义三个正交扰动轴做单参数扫描，定义各轴的 robustness。这些设计比单个数字更有复用价值。

口径与限制

89% 是厂商在特定任务、特定模型规模、特定扰动设置下报告的相关性，不是普适常数。相关性会随任务、渲染保真度、接触复杂度、真机标定而变化。正确用法是把它当成「值得验证的假设」，而不是「可以直接写进验收标准的结论」。

代价 范围很宽，深度待验证：刚体、软体、MPM、布料、渲染、编译器、评估全都要做，任何一层都可能有短板。相关性是任务相关的，89% 不能直接搬到你的任务；需要自己做 sim 与 real 的校准。它不是 Isaac / PhysX 的 drop-in，资产、传感器、ROS、部署链路都要重新接。名称容易混淆：开源 Genesis（2024-12）与 Genesis World 1.0（2026-05）、Genesis AI 的 GENE 模型是不同事物，引用时要核对主体。代码 Apache-2.0，但 Genesis AI 是公司，长期治理、路线图与支持策略需要观察。

4 横向比较：位置、能力与选择

4.1 先比位置，再比能力

表 2 只比较「位置与接口」，不比较性能。它回答的是：这个项目主要替谁付账，向上向下分别接什么。

4.2 能力与约束

表 3 把容易混淆的能力放在一起。需要强调：这里的「可微」指该栈是否提供可用的可微物理路径，不代表所有任务都能直接求梯度；「生态成熟度」是主观判断，不是量化指标。

表 2 六个项目的位置与接口

项目	主层	架构关键词	主要接口 / 典型用户
Isaac Lab	L4	多后端工厂、manager/direct、OpenUSD、ProxyArray、kit-less	向上：任务/训练/部署；向下：PhysX/Newton/OVPhysX；需要完整工作台的团队
MuJoCo / Playground	L5/L2	MJCF、MJX、MJWarp、batch renderer、零样本 sim2real	向上：mjlab/UniLab/Newton 的物理基座；向下：CPU/GPU；做 sim2sim 与快速 baseline 的研究者
mjlab	L2	manager API、MjSpec→MjModel→MjWarp、RSL-RL	向上：任务与环境；向下：MuJoCo Warp；人形/四足快速迭代团队
UniLab	L3	CPU 采样 + GPU 学习、统一运行时、SimBackend 契约	向上：算法与任务；向下：MuJoCoUni/Motrix/多种 GPU 后端；关注端到端吞吐与跨平台的团队
Newton	L5	Warp、MuJoCo Warp、OpenUSD、可微、多物理、LF 治理	向上：Isaac Lab/自定义栈；向下：Warp/CUDA；做复杂物理与可微的研究者
Genesis World	L1	四层栈、Nyx、Quadrants、统一多物理、real-to-sim	向上：评估与数据决策；向下：多后端编译；做基础模型评估的团队

表 3 能力与约束对比

项目	并行 / 执行	渲染	可微	许可 / 硬件	主要限制
Isaac Lab	GPU 并行 + 多 GPU / 多节点	Isaac RTX / OVRTX / Newton	部分	BSD-3 / NVIDIA	重、迁移成本、生态耦合
MuJoCo / Playground	GPU（MJX/Warp）+ CPU	原生 viewer / batch renderer	部分	Apache-2.0 / 广	场景与真实感受限、部署自接
mjlab	GPU（MJWarp），示例 4096 envs	MuJoCo / viser	无	Apache-2.0 / NVIDIA	功能面窄、生态年轻
UniLab	CPU 采样 + GPU 学习	Viser / Motrix	无	Apache-2.0 / CUDA / ROCm / MPS / XPU	仅刚体、仿真主导、单机结论
Newton	GPU（Warp）	Newton GL / USD / RTX	是	Apache-2.0 / Turing+	年轻、求解器差异、生态少
Genesis World	GPU 并行 + 异构环境	Nyx / Luisa / Pyrender	是	Apache-2.0 / 多后端	范围宽、相关性待验证、迁移成本

4.3 决策：先看最贵的返工项

选型不应该从「哪个框架功能多」开始，而应该从「当前最贵的返工项」开始。图 2 给出一个简化的决策图：先诊断症状，再选层级，最后才选项目。这个顺序很重要，因为同一个症状可能由不同层引起：「训练慢」可能是引擎慢，也可能是学习器在等数据；「真机不行」可能是物理不准，也可能是评估不可信。

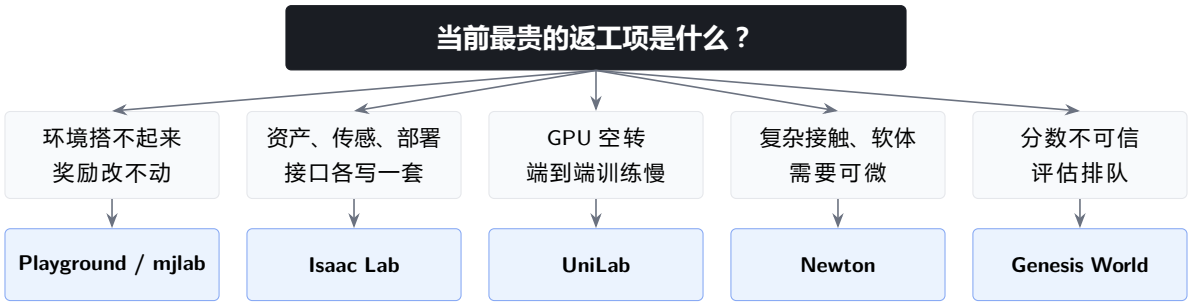


图 2 按「最贵返工项」选层的简化决策图。实际项目往往同时存在多个瓶颈，应先用日志与时间分解定位主因。

可复用结论

选型的第一问不是「哪个框架最强」，而是「我的实验时间主要浪费在哪一层」。如果答不出来，先做一次时间分解：环境搭建、训练、sim2sim、评估、真机调试各占多少。

5 训练栈的系统经济学

5.1 返工从哪里来

把返工拆开，至少有四个来源，对应四个不同的层：

1. **语义错误**：observation 漏了关键量、reward 有漏洞、termination 太早或太晚。这是 L2 入口层的问题，表现为「策略学出奇怪行为」或「改了半天没效果」。
2. **引擎差异**：接触模型、摩擦锥、执行器延迟、求解器收敛不同。这是 L5 引擎层的问题，表现为「仿真里能跑，真机/另一个引擎里不行」。
3. **评估失真**：训练与评估同分布、扰动覆盖不足、相关性未校准。这是 L1 评估层的问题，表现为「分数涨了，真机没涨」。
4. **接口摩擦**：资产、传感器、部署脚本各写一套，团队之间靠口头同步。这是 L4 平台层的问题，表现为「一个人改的东西覆盖了另一个人的工作」。

这四类返工对应四个不同的「最佳工具」：Playground/mjlab 降低第一类，MuJoCo/Newton 降低第二类，Genesis 降低第三类，Isaac Lab 降低第四类。UniLab 比较特殊，它降低的是「等

待」——不是错误，而是空转。

5.2 time-to-robot 的分解

把式 (1) 展开，可以看到每个项目主要压缩哪一项（表 4）。

表 4 项目与主要压缩项

项目	主要压缩项	可观测指标
MuJoCo Playground	t_{task} + 首次调试	从安装到第一个可运行策略的时间
mjlab	t_{task} + 奖励迭代周期	改一次 reward 到看到结果的时间
Isaac Lab	集成 + 部署返工	从训练到真机的返工次数、跨团队接口冲突
UniLab	t_{train} 的墙钟	端到端训练时间、学习器利用率
Newton	长期物理保真度成本	复杂物理覆盖、sim2real gap、可微优化收益
Genesis World	$t_{validate}$ + 评估吞吐	评估周期、sim–real 相关性、失败归因速度

一个常见的误判是：把 t_{train} 当成全部。假设一个实验的训练从 40 分钟降到 10 分钟，但环境搭建要 2 天、评估排队要 1 天、真机调试要 3 天，那么训练时间减少 30 分钟，对整条链路的影响很小。反过来，如果环境搭建从 2 天降到 2 小时，即使训练时间不变，研究节奏也会明显变快。这就是 Playground 与 mjlab 的价值，也是 Isaac Lab 的「重」在某些阶段不划算的原因。

5.3 评估有效性：分数为什么会骗人

评估的核心不是「跑一个 benchmark」，而是「让分数和真实能力对齐」。这里有两个独立的偏差来源。

分布偏差。如果训练和评估都来自同一个仿真分布，模型可以通过拟合仿真器的特定实现来提升分数，而不是提升真实能力。Genesis 的「真实数据训练、仿真评估」是对这个偏差的直接回应：训练分布来自真机，评估分布来自仿真，两者刻意分离。

覆盖偏差。如果评估任务、物体、扰动不够多样，分数只能说明「在这几个任务上还行」。Genesis 的视觉、行为、语义三个正交扰动轴，是一种把覆盖显式化的做法；Isaac Lab 的域随机化与多任务，是另一种。两者不冲突：一个管训练时的覆盖，一个管评估时的覆盖。

判断

评估不是训练之后的「验收」，而是训练之前的「测量仪器」。仪器不准，优化方向就会错。

5.4 接口稳定性与锁定

训练栈的长期成本，很大一部分来自「换掉一个组件要改多少东西」。这可以用三个锁定向量描述：

1. **资产格式**：USD 还是 MJCF/URDF？换格式往往意味着重新做碰撞体、材质、关节限制。
2. **物理后端**：PhysX、MuJoCo、Newton、Genesis 的接触与执行器模型不同，换后端往往要重新调参。
3. **部署 SDK**：真机通信、状态估计、安全逻辑通常与厂商绑定，换机器人比换仿真器更难。

对应的「反锁定」策略是：把任务语义放在稳定的抽象层（manager terms、Hydra 配置、Gymnasium 接口），把物理后端当作可替换实现（Isaac Lab 的 `physics=` 预设、UniLab 的 `SimBackend` 契约），把评估做成独立于训练栈的模块（Genesis 式评估，或自建 eval harness）。这也是本文认为「manager API 成为事实标准」比「某个引擎赢」更重要的原因。

6 组合路线：不要只押一个名字

6.1 三套组合

表 5 按团队形态的组合路线

团队形态	研究前段	主训练 / 验证	评估与回流
小团队 / 个人	Playground 或 mjlab	MuJoCo sim2sim；需要时再上 Isaac Lab	自建真机日志 + 简单相关性检查
人形 / 全身控制	mjlab 做快速消融	Isaac Lab（PhysX + Newton 双后端）	Genesis 或自建 real-to-sim 校准；UniLab 优化吞吐
基础模型 / VLA	真实数据 + 仿真场景	Isaac Lab / Newton 做后训练与物理增强	Genesis 式分布分离、扰动轴测试、失败样本回流

三套组合的共同点是：**研究前段要轻，主训练要稳，评估要独立**。把三者混在一个框架里，短期方便，长期会让「换任何一层」都变成大工程。

6.2 迁移路径

从 Playground 到 mjlab。当环境数量变多、reward 需要长期维护时，把 Playground 的单一任务重写成 mjlab 的 manager terms。物理后端仍然是 MuJoCo 系，资产和观测语义可以复用。

从 mjlab 到 Isaac Lab。当任务需要多模态传感、复杂场景、USD 资产、部署工具时，把 mjlab 的任务语义映射到 Isaac Lab 的 manager/direct 工作流。这一步的成本主要在资产与传感器，而不是算法；建议先做「同一任务、同一 reward、两个引擎」的对照，再迁移训练规模。

从 PhysX 到 Newton。先选一个支持双后端的任务，用相同 checkpoint 做策略迁移验证；如果行为差异可解释（接触、执行器、求解器），再扩大范围。不要把「能加载」当成「等价」。

从自建栈到 UniLab。当瓶颈明确是「学习器等数据 / 采样器等参数」时，把任务接到 `SimBackend` 契约上，先用一个任务验证端到端收益，再考虑多算法、多后端。

6.3 反模式

1. **一开始就上最重的平台。**在 reward 还没跑通时，把时间花在 USD、传感器、集群配置上，是最常见的浪费。先用轻量入口把问题定义清楚。
2. **只看峰值 FPS。**物理步进快，不代表端到端快；学习器利用率、数据搬运、评估排队都可能吃掉收益。
3. **训练与评估共用一个仿真器、一个分布。**分数会虚高；至少要做一次跨引擎或跨分布的检查。
4. **把厂商数字当验收标准。** 3–10×、89%、单命令安装都是特定条件下的结果，不是你的任务上的承诺。
5. **绑定单一后端。**任务代码直接调用某个引擎的私有 API，会让 sim2sim 和迁移变得极贵。
6. **没有失败日志回流。**真机摔倒、打滑、抖动如果只留在聊天记录里，下一轮训练不会变好。

6.4 一个最小可行闭环

如果要从零搭一个「能持续返工」的训练栈，最小闭环是：

1. **一个可读的任务定义：**observation、action、reward、termination 能被人读懂、能 diff。
2. **一个快速训练入口：**单卡、分钟级、可复现。
3. **一个独立 sim2sim：**至少两个物理实现，能解释差异。
4. **一个独立评估：**与训练分布分离，有扰动轴，有相关性检查。
5. **一条真机日志回路：**失败样本能回到任务定义与评估集。

这五件事不需要同一个框架完成。它们需要的是清晰的接口和稳定的语义。

7 判断与观察清单

7.1 六个判断

1. **分层是真实的，不是营销词。**六个项目的主要价值主张分别落在评估、入口、运行时、平台、引擎、计算，把它们放进同一张「谁最强」的表里，问题本身就被问错了。
2. **竞争单位从 FPS 转向每次有效验证的返工成本。**这解释了为什么更快的仿真器不一定让研究更快，也解释了为什么「轻量入口」和「重型平台」可以同时存在。

3. **三次解耦最重要：平台与引擎解耦、运行时与仿真器解耦、评估与训练解耦。** Isaac Lab 3.0 的多后端、UniLab 的异构运行时、Genesis 的评估产品化，分别对应这三次解耦。
4. **manager API 正在成为事实标准。** Isaac Lab、mjlab、UniLab 都采用类似的 term 组织；这意味着任务语义的可迁移性在提高，品牌锁定的重要性在下降。
5. **物理引擎的竞争会从「刚体快不快」转向「多物理 + 可微 + 治理」。** Newton 的位置在这里；它短期不替代谁，但会影响未来复杂任务的底层写法。
6. **评估会成为独立的一层。**当模型规模变大、真机评估变贵，「怎么知道它真的变好了」会从附属问题变成核心问题；Genesis 的叙事是这一趋势的早期信号。

7.2 未来 12 个月看什么

表 6 观察清单：信号比承诺更重要

项目	看什么	触发采用的信号
Isaac Lab	Newton 后端的任务覆盖、PhysX↔Newton 迁移的稳定性、kit-less 的安装体验	你的目标任务在 Newton 预设里可用，且迁移差异可解释
MuJoCo / Playground	MJWarp 与 MJX 的算子/性能收敛、视觉任务覆盖、真机示例	你的任务有可复现的 sim2real 示例，且单卡可跑
mjlab	传感器/地形/多物理覆盖、第三方任务生态、Unitree 之外的厂商采用	你的机型有官方或社区任务，且维护活跃
UniLab	多算法/多后端的稳定性、视觉任务扩展、集群规模验证	你的瓶颈是端到端吞吐，且任务是刚体/仿真主导
Newton	求解器覆盖、Isaac Lab 集成深度、文档与 API 稳定性、多物理案例	你需要软体/布料/颗粒/可微，且能接受早期采用成本
Genesis World	相关性数字的独立复现、Nyx/Quadrants 的可用性、评估方法论的工具化	你要做基础模型评估，且愿意自建校准

7.3 风险

整合风险。六个项目背后的资源、治理、商业目标不同，未来可能出现合并、停滞或路线转向。不要把任务语义绑死在某个项目的私有 API 上。

复现风险。厂商报告的加速比与相关性，往往依赖特定硬件、任务与配置。独立复现的成本被低估，是评估「新框架」时最常见的坑。

治理风险。 Linux Foundation 治理、Apache-2.0 许可、公司主导的开发，三者并不等价。许可允许你使用，不代表路线图会照顾你的需求。

硬件风险。 GPU 代际、驱动版本、CUDA/ROCm/Metal 的差异，会让「同一套代码」在不同机器上表现不同。跨平台是设计目标，不是免费属性。

7.4 对无人机读者的映射

本仓库的另一条线是 UAV 仿真（Gazebo、Isaac Sim、AirSim、Flightmare、RflySim、XTDrone）。同样的分层逻辑可以套用：Gazebo / Isaac Sim 更像 L4 平台，PX4 SITL / HITL 更像 L3 运行时与硬件在环，Flightmare / AirSim 更像 L2 入口与渲染，而真实飞行日志与飞行评估则是 L1。无人机领域更早遇到「HIL 与真机安全」的问题，机器人领域更早遇到「多模态与全身接触」的问题；两边的方法论可以互相借。

8 结论

回到开头的问题：有了 Isaac Lab 和 MuJoCo，为什么还会出现 mjlab、UniLab、Newton、Genesis？

因为机器人学习最贵的地方，已经不是「单次仿真跑多快」，而是「整条训练与评估闭环要返工多少次」。Isaac Lab 降低平台层的接口摩擦，MuJoCo 与 Playground 降低物理验证与首次尝试的成本，mjlab 降低任务组织的长期维护成本，UniLab 降低等待与空转，Newton 降低复杂物理与可微优化的底层成本，Genesis 降低评估与置信建立的成本。它们不是在同一张牌桌上抢同一个位置，而是在把牌桌加厚。

判断

单独看，谁都不是 Sim2Real 的答案。真正的答案，是一套能把真实失败持续带回训练系统的机器人训练栈。工具会变，接口会变，物理引擎也会变；能让「仿真里学到的东西」经得起真机失败反复拷问的闭环，才是长期资产。

A 术语表

B 资料与口径

本文的资料核对日期为 2026 年 10 月 28 日。主要来源包括：

- Isaac Lab：论文 [1]、3.0 GA release notes [2]、官方文档 [3]。
- MuJoCo / Playground：论文 [4]、项目页 [5]、MuJoCo Warp [6]。
- mjlab：论文 [7]、代码仓库、Unitree RL Mjlab [8]。

术语	含义
MJCF	MuJoCo 的 XML 模型格式，描述刚体、关节、执行器、传感器与接触参数
MJX	MuJoCo 的 JAX 实现，支持 GPU/TPU 批量仿真与自动微分
MJWarp	MuJoCo Warp，基于 NVIDIA Warp 的 GPU 实现
OpenUSD	Universal Scene Description，场景与资产的组合/交换格式
Manager API	把 observation、reward、event、command、curriculum 拆成可组合 term 的环境组织方式
Kit-less	不启动 Omniverse Kit / Isaac Sim，仅用独立库运行 Isaac Lab 工作流
Sim2Sim	在另一个仿真器中验证策略，检查它是否只适配了单一引擎
Real-to-Sim	用真机数据校准/驱动仿真评估，强调评估分布与训练分布分离
VBD	Vertex Block Descent，一种基于顶点的物理求解器
MPM	Material Point Method，适合颗粒、雪、沙、流体等材料

- UniLab：论文 [9]、代码仓库 [10]、论文项目页。
- Newton：代码仓库 [11]、Linux Foundation 公告 [12]、Isaac Lab 3.0 文档。
- Genesis World：官方博客 [13]、代码仓库 [14]、PyPI 页面。

需要特别说明的口径：

- UniLab 的 3–10× 是论文在受控单 CPU / 单 GPU 工作站上的端到端结果，不是所有任务、所有硬件上的保证。
- Genesis 的 89% 相关性与「166–200 小时到约 30 分钟」是厂商/公开报道口径，任务与模型规模特定，需独立验证。
- mjlab 的「单命令安装」指 demo 与安装路径，不代表所有任务、所有机型都零配置。
- Isaac Lab 的 3.0 版本信息以 2026-10-20 的 GA release 为准；后续版本可能改变 API 与后端支持矩阵。

参考文献

[1] NVIDIA. Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning. arXiv:2511.04831, 2025. https://arxiv.org/abs/2511.04831

[2] NVIDIA. Isaac Lab 3.0 General Availability. GitHub release, 2026-10-20. https://github.com/isaac-sim/IsaacLab/releases/tag/v3.0.0

[3] The Isaac Lab Project Developers. Isaac Lab 3.0 Documentation: Backends and Presets, Physics Backends, Release Notes. https://isaac-sim.github.io/IsaacLab/

[4] Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder A. Kahrs, Carmelo Sferrazza, Yuval Tassa, Pieter Abbeel. MuJoCo Playground. arXiv:2502.08844, 2025. https://arxiv.org/abs/2502.08844

[5] Kevin Zakka et al. MuJoCo Playground Project Page. https://playground.mujoco.org/

[6] Google DeepMind. MuJoCo Warp: GPU-Accelerated MuJoCo with NVIDIA Warp. https://github.com/google-deepmind/mujoco_warp

[7] Kevin Zakka, Qiayuan Liao, Brent Yi, Louis Le Lay, Koushil Sreenath, Pieter Abbeel. mjlab: A Lightweight Framework for GPU-Accelerated Robot Learning. arXiv:2601.22074, 2026. https://arxiv.org/abs/2601.22074

[8] Unitree Robotics. Unitree RL Mjlab: Train, Play, Sim2Real on MuJoCo. https://github.com/unitreerobotics/unitree_rl_mjlab

[9] Yufei Jia, Zhanxiang Cao, Mingrui Yu, Heng Zhang, Shenyu Chen, Dixuan Jiang, et al. UniLab: A Heterogeneous Architecture for Robot RL Beyond GPU-Dominant Paradigms. CoRL 2026; arXiv:2605.30313. https://arxiv.org/abs/2605.30313

[10] UniLab Contributors. UniLab: Contract-Driven Infrastructure for Robot Learning Across Physics Backends and Hardware. https://github.com/unilabsim/UniLab

[11] Newton Physics Project. Newton: GPU-Accelerated Physics Simulation Engine. https://github.com/newton-physics/newton

[12] The Linux Foundation. Linux Foundation Announces Contribution of Newton by Disney Research, Google DeepMind, and NVIDIA to Accelerate Open Robot Learning. 2025. Linux Foundation press release.

[13] Genesis AI Team. The Role of Simulation in Scalable Robotics, Genesis World 1.0, and the Path Forward. Genesis AI Blog, 2026-05. https://www.genesis.ai/blog/the-role-of-simulation-in-scalable-robotics-genesis-world-10-and-the-path-forward

[14] Genesis Embodied AI. Genesis World: A Simulation Platform for Physical AI. https://github.com/Genesis-Embodied-AI/genesis-world

[15] Emanuel Todorov, Tom Erez, Yuval Tassa. MuJoCo: A Physics Engine for Model-Based Control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[16] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunfei Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, Gavriel State. Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning. arXiv:2108.10470, 2021.

[17] NVIDIA. NVIDIA Warp: A Python Framework for High-Performance Simulation and Spatial Computing. https://github.com/NVIDIA/warp